

**Segurança em Sistemas Operacionais e Redes de
Computadores II**

ISG012-03 - Firewalls de Perímetro e Filtragem Avançada - Teoria

Objetivo

- O objetivo desta aula é capacitar os estudantes a compreenderem e implementarem arquiteturas de segurança de perímetro utilizando firewalls modernos. Ao final desta sessão, o aluno será capaz de diferenciar os tipos de firewalls (packet filtering, stateful inspection e application layer), projetar uma zona desmilitarizada (DMZ) segura, configurar regras avançadas de NAT (SNAT e DNAT) utilizando nftables, e aplicar conceitos de segmentação de rede para proteger ativos corporativos críticos contra acessos não autorizados provenientes da Internet.

Objetivo

- Diferenciar arquiteturas de firewalls (packet filter, stateful, application layer)
- Projetar e implementar uma DMZ segura
- Configurar NAT avançado (SNAT/DNAT) com nftables
- Aplicar segmentação de rede para proteção de perímetro
- Compreender logging e monitoramento de firewalls

Introdução aos Firewalls de Perímetro

- Firewalls de perímetro constituem a primeira linha de defesa entre uma rede organizacional e a Internet. Diferentemente dos firewalls de host, que protegem máquinas individuais, os firewalls de perímetro controlam o tráfego entre redes distintas, tipicamente posicionados na borda da infraestrutura. Sua função principal é implementar a política de segurança de acesso, determinando quais conexões podem entrar, sair ou atravessar os limites da rede organizacional. A eficácia de um firewall de perímetro depende de sua correta colocação topológica, da precisão de suas regras de filtragem e da capacidade de inspeção de tráfego em múltiplas camadas do modelo OSI.

Introdução aos Firewalls de Perímetro

- Posicionamento na borda da rede (edge/border)
- Controle de tráfego entre redes de diferentes níveis de confiança
- Implementação da política de segurança de acesso
- Ponto único de entrada/saída monitorado
- Base para arquiteturas Defense in Depth

Arquiteturas de Firewall

- As arquiteturas de firewall evoluíram significativamente desde os simples filtros de pacotes da primeira geração até os Next-Generation Firewalls (NGFW) atuais. O packet filtering firewall opera na camada 3 do modelo OSI, analisando apenas cabeçalhos IP e portas TCP/UDP, sendo rápido mas limitado. O stateful inspection firewall adiciona o conceito de tabela de estados, rastreando conexões ativas e permitindo decisões contextuais. O application layer firewall (proxy) inspeciona o conteúdo dos pacotes na camada 7, oferecendo segurança superior ao custo de maior latência. Cada arquitetura apresenta trade-offs entre performance e nível de segurança.

Arquiteturas de Firewall

- Packet Filter (1ª geração): Camada 3, análise de IP/porta, stateless
- Stateful Inspection (2ª geração): Camadas 3-4, tabela de estados, tracking de sessões
- Application Layer (3ª geração): Camada 7, proxy, inspeção profunda de conteúdo
- NGFW: Integração de múltiplas funções (IPS, VPN, identificação de aplicações)
- Trade-off: Performance vs. profundidade de inspeção

Evolução das Arquiteturas de Firewall

- A evolução das arquiteturas de firewall reflete a crescente sofisticação das ameaças cibernéticas. Os primeiros filtros de pacotes (packet filtering) eram stateless, analisando cada pacote isoladamente sem contexto de conexão, o que os tornava vulneráveis a ataques de spoofing. A segunda geração introduziu o stateful inspection, mantendo uma tabela de estado que registra todas as conexões ativas (5-tuple: IP origem, IP destino, protocolo, porta origem, porta destino), permitindo que o firewall entenda se um pacote pertence a uma sessão estabelecida. A terceira geração expandiu a inspeção para a camada de aplicação, analisando payloads HTTP, FTP e outros protocolos, possibilitando bloquear malware e conteúdo malicioso que poderia passar despercebido em firewalls tradicionais.

Evolução das Arquiteturas de Firewall

- **1ª Geração:** Packet Filter - Rápido, sem estado, vulnerável a spoofing
- **2ª Geração:** Stateful Inspection - Contexto de conexão, tabela de estados, bloqueia spoofing
- **3ª Geração:** Application Layer/Proxy - Inspeção profunda, controle de conteúdo, alta latência
- **NGFW:** Deep Packet Inspection (DPI), identificação de usuários, integração com AD
- **Tendência:** Convergência de funções de segurança em appliances unificados

Evolução das Arquiteturas de Firewall

1ª GERAÇÃO

Packet Filter

(Filtro de Pacotes)

Camada 3 (Rede)

- IP Origem/Destino
- Porta TCP/UDP
- Protocolo (ICMP)

Limitações:

- Sem estado de conexão
- Vulnerável a spoofing

RÁPIDO

Processamento mínimo
Alta performance

2ª GERAÇÃO

Stateful Inspection

(Inspeção de Estado)

Camadas 3-4 (Rede/Transporte)

- Tabela de Estado
- Tracking de sessões
- 5-tuple matching

Vantagens:

- Contexto de conexão
- Bloqueia spoofing
- Regras simplificadas
- Retorno automático

EQUILIBRADO

Segurança + Performance
Uso generalizado

3ª GERAÇÃO / NGFW

Application Layer / NGFW

(Camada de Aplicação)

Camada 7 (Aplicação)

- Deep Packet Inspection (DPI)
- Identificação de aplicativos
 - Inspeção SSL/TLS
 - IDS/IPS integrado

Recursos Avançados:

- Controle de usuários (AD/LDAP)
 - Filtragem de conteúdo
 - Prevenção de ameaças
- Sandbox e análise de comportamento

ALTO PROCESSAMENTO

Máxima segurança
Requer hardware robusto

Pacote:

Cabeçalho IP

Cabeçalho TCP

Payload (Dados)

← Packet Filter analisa apenas o IP | Stateful analisa IP+TCP | NGFW analisa tudo

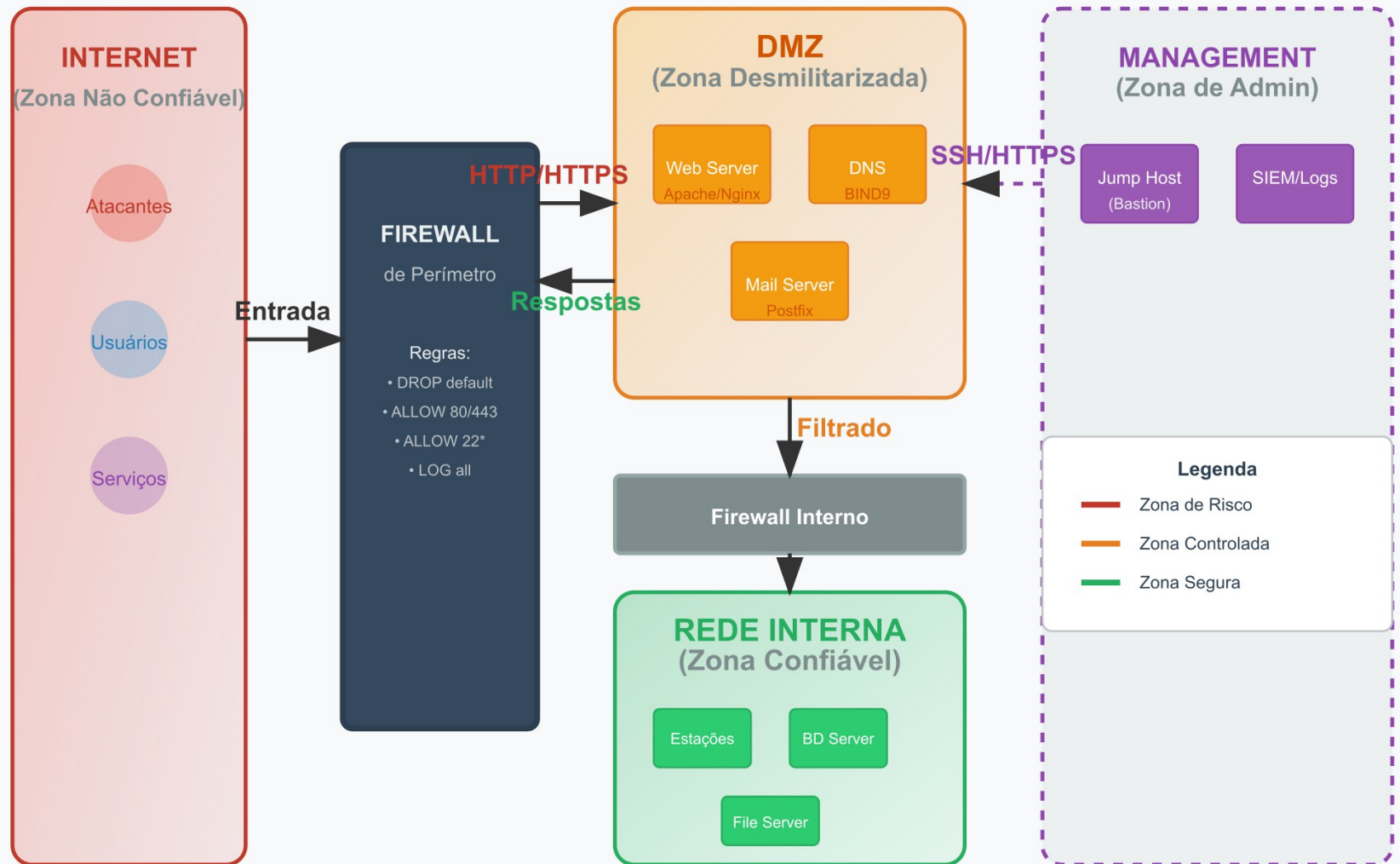
Conceito de DMZ (Zona Desmilitarizada)

- A Zona Desmilitarizada (DMZ - Demilitarized Zone) é uma sub-rede física ou lógica que expõe serviços externos de uma organização à Internet, isolando-os da rede interna corporativa. A DMZ funciona como uma área intermediária de segurança, onde residem servidores que precisam ser acessíveis externamente (web, DNS, email), mas que não devem ter acesso direto aos ativos internos sensíveis. A arquitetura típica utiliza dois firewalls: um externo (perímetro) que protege a DMZ da Internet, e um interno que protege a rede corporativa da DMZ. Se um servidor na DMZ for comprometido, o atacante ainda enfrenta uma barreira adicional para alcançar a rede interna.

Conceito de DMZ (Zona Desmilitarizada)

- Rede intermediária entre Internet e rede interna
- Hospeda serviços públicos (web, DNS, email, FTP)
- Isolamento de ativos críticos internos
- Arquitetura de duplo firewall ou tri-homed firewall
- Princípio: comprometimento da DMZ $\neq$ acesso à rede interna

Arquitetura de Firewall de Perímetro com DMZ



Arquitetura de Firewall de Perímetro com DMZ

- A arquitetura de perímetro com DMZ estabelece três zonas de segurança distintas: a zona não confiável (Internet), a zona controlada (DMZ) e a zona confiável (rede interna). O firewall de perímetro (externo) filtra tráfego entre Internet e DMZ, tipicamente permitindo apenas portas específicas (80, 443, 25, 53) para servidores públicos. O firewall interno controla o tráfego entre DMZ e rede interna, sendo mais restritivo e permitindo apenas conexões iniciadas internamente ou respostas a solicitações legítimas. Uma zona adicional de gerenciamento (management) pode ser implementada para acesso administrativo seguro via jump hosts (bastion hosts), isolando o acesso administrativo do tráfego operacional.

Arquitetura de Firewall de Perímetro com DMZ

- Zona Não Confiável (Internet): Acesso irrestrito, maior risco
- Zona Controlada (DMZ): Serviços públicos, acesso limitado
- Zona Confiável (Internal): Ativos críticos, acesso restrito
- Firewall Perímetro: Entre Internet e DMZ, regras permissivas controladas
- Firewall Interno: Entre DMZ e Internal, regras restritivas
- Zona de Gerenciamento: Acesso administrativo isolado (jump host)

Design de DMZ e Posicionamento de Serviços

- O design efetivo de uma DMZ requer planejamento cuidadoso do posicionamento de serviços baseado em sua exposição necessária e criticidade. Servidores web públicos são colocados na DMZ frontal, acessíveis diretamente da Internet. Servidores de aplicação que processam dados sensíveis podem residir em uma DMZ de backend ou em uma zona semi-confiável. Bancos de dados e sistemas de arquivos críticos permanecem exclusivamente na rede interna, sem acesso direto da DMZ. O princípio do menor privilégio determina que servidores na DMZ devem ter apenas as portas necessárias abertas e comunicação restrita com a rede interna, tipicamente através de protocolos específicos em portas predefinidas.

Design de DMZ e Posicionamento de Serviços

- DMZ Frontal: Web servers, proxies reversos, DNS externos
- DMZ de Backend: Application servers, middleware, serviços de integração
- Rede Interna: Bancos de dados, file servers, AD, sistemas críticos
- Princípio do menor privilégio: Apenas portas necessárias, comunicação mínima
- Segmentação vertical: Múltiplas DMZs para diferentes níveis de sensibilidade
- Jump Hosts: Pontos de acesso administrativo únicos e monitorados

Stateful vs Stateless Inspection

- A distinção entre inspeção stateful e stateless é fundamental para compreender as capacidades de um firewall. Firewalls stateless (packet filters tradicionais) analisam cada pacote individualmente, baseando decisões apenas em informações estáticas do cabeçalho (IP origem/destino, porta, protocolo), sem memória de conexões anteriores. Firewalls stateful mantêm uma tabela de estado (state table) que registra todas as conexões ativas, permitindo identificar se um pacote pertence a uma sessão estabelecida, é uma nova conexão ou um pacote inválido. Essa capacidade permite que firewalls stateful bloqueiem ataques de spoofing e session hijacking, além de simplificar as regras (não é necessário permitir explicitamente o tráfego de retorno).

Stateful vs Stateless Inspection

- Stateless: Sem memória de estado, cada pacote analisado isoladamente
- Stateful: Mantém tabela de conexões ativas (state table)
- 5-tuple matching: IP src/dst, protocol, port src/dst para identificar sessões
- Vantagens stateful: Bloqueia spoofing, simplifica regras, permite tráfego de retorno automático
- Timeout de sessão: Estados expiram após período de inatividade
- Uso moderno: Stateful é padrão; stateless usado em data centers de alta performance

Tabela de Estados (State Table)

- A tabela de estados é a estrutura de dados central de um firewall stateful, mantendo registros de todas as conexões de rede ativas que passaram pelo dispositivo. Cada entrada na tabela tipicamente contém o 5-tuple (endereço IP de origem e destino, portas de origem e destino, protocolo), o estado atual da conexão TCP (SYN_SENT, ESTABLISHED, TIME_WAIT), timestamps de última atividade, e flags de controle. Quando um pacote chega, o firewall primeiro consulta a tabela de estados; se houver correspondência, o pacote é permitido sem nova avaliação de regras, otimizando performance. Se não houver correspondência, o pacote é avaliado contra as regras de firewall, e se permitido, uma nova entrada é criada na tabela.

Tabela de Estados (State Table)

- Estrutura: Hash table ou árvore balanceada para busca rápida
- Campos: 5-tuple, estado TCP, contadores de pacotes/bytes, timestamp
- Estados TCP: NONE, SYN_SENT, SYN_RECV, ESTABLISHED, FIN_WAIT, CLOSE_WAIT, LAST_ACK, TIME_WAIT, CLOSE
- Estados UDP/ICMP: Simulados (UDP tem estado "virtual" baseado em timeout)
- Timeout dinâmico: TCP estabelecido (horas), TCP SYN (minutos), UDP (segundos)
- Limitações: Consumo de memória RAM proporcional ao número de conexões

Network Address Translation (NAT)

- Network Address Translation (NAT) é uma técnica essencial em firewalls de perímetro que permite múltiplos hosts em redes privadas (RFC 1918: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) compartilharem endereços IP públicos para comunicação com a Internet. O NAT opera modificando os cabeçalhos IP de pacotes em trânsito, substituindo endereços privados por públicos (ou vice-versa). Além da conservação de endereços IPv4, o NAT proporciona uma camada adicional de segurança através da ocultação da topologia interna da rede, dificultando que atacantes externos mapeiem diretamente a infraestrutura interna. O NAT é tipicamente implementado no firewall de perímetro, no ponto de saída da rede para a Internet.

Network Address Translation (NAT)

- Conservação de IPs: Múltiplos hosts privados compartilham IP público
- Ocultação de topologia: Rede interna invisível da Internet
- RFC 1918: Endereços privados não roteáveis na Internet pública
- Stateful NAT: Rastreia conexões NAT na tabela de estados
- Segurança adicional: Barreira contra scanning direto de hosts internos
- Limitações: Quebra conexões end-to-end, complica troubleshooting

Tipos de NAT: SNAT (Source NAT)

- Source NAT (SNAT), também conhecido como mascaramento (masquerading) quando o IP de origem é dinâmico, é o tipo de NAT utilizado quando hosts internos iniciam conexões para a Internet. O SNAT altera o endereço IP de origem dos pacotes que saem da rede privada, substituindo-o pelo endereço IP público do firewall. No nftables, o SNAT é implementado na chain POSTROUTING (após a decisão de roteamento), pois a alteração do endereço de origem deve ocorrer depois que o pacote já foi roteado para a interface externa. O SNAT pode ser estático (mapeamento fixo 1:1) ou dinâmico (masquerade, usando o IP atual da interface), sendo o masquerade preferido quando o IP público é atribuído via DHCP.

Tipos de NAT: SNAT (Source NAT)

- Função: Altera IP de origem de pacotes saindo da rede
- Chain: POSTROUTING (após roteamento, antes de sair pela interface)
- Sintaxe nftables: snat to IP ou masquerade (para IP dinâmico)
- Uso principal: Acesso à Internet a partir de rede privada
- SNAT Estático: Um IP interno → Um IP externo fixo
- Masquerade: Múltiplos IPs internos → IP dinâmico da interface externa
- Port Address Translation (PAT): Múltiplos hosts usam mesmo IP com portas diferentes

Tipos de NAT: DNAT (Destination NAT)

- Destination NAT (DNAT) é utilizado quando tráfego da Internet precisa acessar servidores localizados na rede interna ou DMZ. O DNAT altera o endereço IP de destino dos pacotes que chegam do exterior, redirecionando-os para endereços privados internos. Esta técnica é fundamental para publicar serviços internos (web, email, SSH) na Internet sem expor diretamente os servidores. No nftables, o DNAT é implementado na chain PREROUTING (antes da decisão de roteamento), pois a alteração do destino deve ocorrer para que o pacote seja corretamente roteado para a rede interna. O DNAT pode redirecionar para diferentes portas (port forwarding), permitindo, por exemplo, que a porta 2222 externa acesse a porta 22 (SSH) interna.

Tipos de NAT: DNAT (Destination NAT)

- Função: Altera IP de destino de pacotes entrando da Internet
- Chain: PREROUTING (antes do roteamento, logo após entrada na interface)
- Sintaxe nftables: `dnat to IP:porta`
- Uso principal: Publicação de servidores internos (port forwarding)
- Port Forwarding: Redirecionamento de portas (ex: 2222 externo → 22 interno)
- Load Balancing: DNAT para múltiplos backends (round-robin)
- Segurança: Permite ocultar servidores reais atrás de IPs públicos

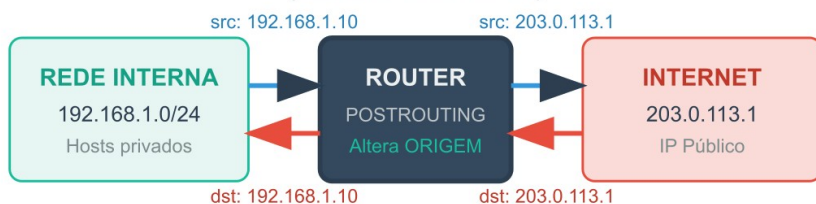
Comparação SNAT vs DNAT

- SNAT e DNAT são operações complementares que atuam em direções opostas do fluxo de tráfego. O SNAT modifica o endereço de origem e é aplicado na chain POSTROUTING, sendo utilizado para tráfego de saída (outbound), permitindo que múltiplos hosts internos acessem a Internet usando um único IP público. O DNAT modifica o endereço de destino e é aplicado na chain PREROUTING, sendo utilizado para tráfego de entrada (inbound), permitindo que servidores internos sejam acessíveis externamente. Ambos requerem que o firewall tenha habilitado o IP forwarding (roteamento entre interfaces) e que as regras de filtragem (filter table) permitam o tráfego encaminhado (chain FORWARD).

Tipos de NAT: SNAT vs DNAT

SNAT - Source NAT

(Mascaramento / Saída)



Casos de Uso SNAT:

- Acesso à Internet a partir de rede privada (RFC 1918)
- Mascaramento dinâmico (IP DHCP do provedor)
- Balanceamento de saída com múltiplos IPs públicos

DNAT - Destination NAT

(Redirecionamento / Entrada)



Casos de Uso DNAT:

- Publicação de servidores internos na Internet (Port Forwarding)
- Load balancing entre múltiplos servidores backend
- Redirecionamento de portas (ex: 2222 externo → 22 interno)

Comparação SNAT vs DNAT

Característica	SNAT	DNAT
Campo alterado (Source Address)	IP de Origem src → IP público	IP de Destino dst → IP interno
Chain (nftables) Após roteamento	postrouting priority 100	prerouting priority -100
Direção do tráfego Saída da rede	Interno → Externo Outbound	Externo → Interno Inbound
Uso principal Função típica	Mascaramento Acesso web	Port Forwarding Publicar serviço
Comando nftables Sintaxe	snat to IP ou masquerade	dnat to IP:port Redirecionamento
Segurança Impacto	Ocultar rede interna Protege identidade	Expõe serviço Requer hardening

Exemplos de Comandos

```

# SNAT - Mascaramento
nft add rule nat postrouting oifname "eth0" masquerade

# DNAT - Port Forwarding
nft add rule nat prerouting iifname "eth0" tcp dport 80 \
dnat to 192.168.1.10:80

# SNAT Estático (IP fixo)
nft add rule nat postrouting oifname "eth0" snat to 203.0.113.1
  
```

Comparação SNAT vs DNAT

- SNAT: POSTROUTING, altera origem, tráfego outbound (interno → externo)
- DNAT: PREROUTING, altera destino, tráfego inbound (externo → interno)
- SNAT: Mascaramento de múltiplos hosts, acesso à Internet
- DNAT: Publicação de serviços, port forwarding
- Requisito comum: IP forwarding habilitado (`net.ipv4.ip_forward=1`)
- Chain FORWARD: Deve permitir tráfego entre interfaces (filter table)
- Combinação: DMZ tipicamente usa ambos (DNAT para entrada, SNAT para saída)

Fluxo de Processamento no Netfilter/nftables

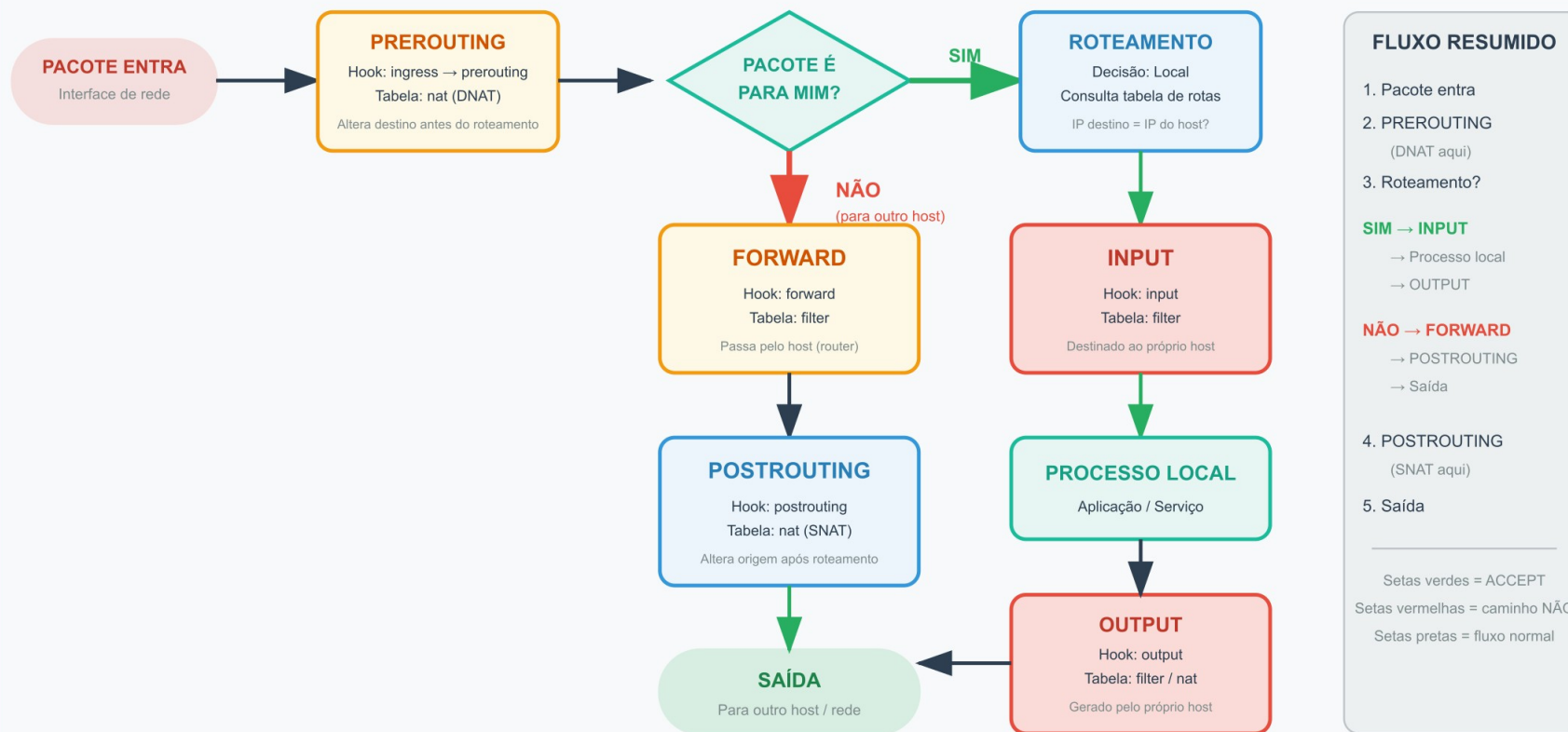
- O framework netfilter no kernel Linux processa pacotes através de uma série de hooks (pontos de interceptação) onde firewalls podem inspecionar e modificar o tráfego. Quando um pacote entra pela interface de rede, passa primeiro pelo hook PREROUTING (onde DNAT é aplicado), depois pela decisão de roteamento (o pacote é para mim ou para outro host?). Se for para outro host, passa pela chain FORWARD e depois POSTROUTING (onde SNAT é aplicado) antes de sair. Se for para o próprio host, passa pela chain INPUT e entrega ao processo local. Pacotes gerados localmente saem pela chain OUTPUT e depois POSTROUTING. Cada hook pode conter múltiplas tabelas (filter, nat, mangle, raw) com chains específicas.

Fluxo de Processamento no Netfilter/nftables

- PREROUTING: Primeiro hook, antes do roteamento (DNAT aqui)
- Decisão de roteamento: Pacote é local ou para encaminhar?
- INPUT: Pacotes destinados ao próprio host
- FORWARD: Pacotes passando pelo host (router/firewall)
- OUTPUT: Pacotes gerados pelo próprio host
- POSTROUTING: Último hook, após roteamento (SNAT aqui)
- Ordem de tabelas: raw → mangle → nat → filter

Fluxo de Processamento de Pacotes no nftables

Como o kernel Linux processa pacotes através das chains e hooks do netfilter



AÇÕES POSSÍVEIS

- ACCEPT — permite
- DROP — descarta (silencioso)
- REJECT — descarta (com aviso)
- LOG — registra no log

Tabelas do nftables



filter: Filtragem de pacotes (accept/drop/reject)
nat: Tradução de endereços (snat/dnat/masquerade)
mangle: Modificação de cabeçalhos | raw: Exceções de tracking

Ordem de processamento das tabelas em cada hook

PREROUTING: raw → mangle → nat → (roteamento)
INPUT: mangle → nat → filter
FORWARD: mangle → filter
OUTPUT: raw → mangle → nat → filter
POSTROUTING: mangle → nat

Tabelas e Chains no nftables

- O nftables organiza as regras de firewall em tabelas (tables), que são coleções de chains (cadeias de regras) associadas a uma família de protocolos (ip, ip6, inet, arp, bridge). A tabela filter é usada para filtragem de pacotes (accept, drop, reject), sendo a mais comum para políticas de segurança básicas. A tabela nat é usada para tradução de endereços (snat, dnat, masquerade, redirect). A tabela mangle permite modificação de cabeçalhos IP (TOS, TTL). A tabela raw é usada para exceções de connection tracking. Cada tabela contém chains que se ligam aos hooks do netfilter (input, output, forward, prerouting, postrouting), permitindo processamento em diferentes pontos do fluxo do pacote.

Tabelas e Chains no nftables

- Famílias: inet (IPv4+IPv6), ip (IPv4), ip6 (IPv6), arp, bridge
- Tabela filter: Filtragem básica (accept, drop, reject, log)
- Tabela nat: Tradução de endereços (snat, dnat, masquerade, redirect)
- Tabela mangle: Modificação de cabeçalhos, marcação de pacotes
- Tabela raw: Exceções de connection tracking (notrack)
- Chains base: input, output, forward, prerouting, postrouting
- Chains definidas pelo usuário: Nomes arbitrários, ligados a hooks específicos

Sintaxe Básica do nftables

- O nftables utiliza uma sintaxe declarativa e unificada que substitui as ferramentas legadas (iptables, ip6tables, arptables, ebtables). A estrutura hierárquica é: tabela → chain → regra. Para criar uma configuração básica, primeiro define-se a tabela (`nft add table inet filter`), depois a chain (`nft add chain inet filter input { type filter hook input priority 0 \; }`), e então as regras (`nft add rule inet filter input tcp dport 22 accept`). A sintaxe suporta expressões complexas, conjuntos (sets) para múltiplos endereços/portas, e mapas para configurações dinâmicas. As regras podem ser persistentes quando salvas em `/etc/nftables.conf` e carregadas com `nft -f /etc/nftables.conf`.

Sintaxe Básica do nftables

- Estrutura: Table → Chain → Rule
- Criação de tabela: `nft add table <família> <nome>`
- Criação de chain: `nft add chain <table> <chain> { type <tipo> hook <hook> priority <n> \; }`
- Adição de regra: `nft add rule <table> <chain> <expressão> <ação>`
- Sets: `{ 22, 80, 443 }` para múltiplos valores
- Persistência: Arquivo `/etc/nftables.conf` carregado via `systemd`
- Visualização: `nft list ruleset` mostra todas as regras ativas

Regras de Filtragem Stateful

- A configuração de regras stateful no nftables aproveita o connection tracking (conntrack) do kernel para identificar o estado das conexões. A expressão ct state permite verificar se um pacote pertence a uma conexão estabelecida (established), relacionada (related), nova (new) ou inválida (invalid). A política recomendada é permitir tráfego established e related (retorno de conexões iniciadas internamente), permitir new para serviços específicos desejados, e dropar invalid. Esta abordagem simplifica drasticamente as regras, pois não é necessário criar regras explícitas para o tráfego de retorno. A ordem das regras é importante: established/related devem vir primeiro para otimização de performance.

Regras de Filtragem Stateful

- Estados conntrack: new, established, related, invalid, untracked
- Regra established: ct state established,related accept (tipicamente primeira regra)
- Regra new: ct state new tcp dport 22 accept (para novas conexões permitidas)
- Regra invalid: ct state invalid drop (bloqueia pacotes malformados)
- Vantagem: Não precisa de regras explícitas para tráfego de retorno
- Performance: Established é processado rapidamente sem avaliação de regras adicionais
- Protocolos sem estado: UDP e ICMP têm estados "virtuais" baseados em timeouts

Logging e Monitoramento de Firewalls

- O logging é essencial para auditoria, troubleshooting e detecção de tentativas de intrusão em firewalls. No nftables, o logging pode ser implementado via ação log (mensagens no kernel log, acessíveis via dmesg ou journalctl) ou counter (contagem de pacotes sem log). É recomendado logar drops de tráfego suspeito, especialmente na chain INPUT, para identificar tentativas de scanning e ataques. O prefixo de log permite identificar a origem da mensagem (log prefix `"DROP-INPUT: "`). Para análise centralizada, os logs podem ser encaminhados via rsyslog para um servidor SIEM. O monitoramento contínuo inclui análise de padrões de tráfego, detecção de anomalias e alertas automatizados para eventos de segurança.

Logging e Monitoramento de Firewalls

- Ação log: log prefix \"descricao: \" envia para kernel log
- Visualização: dmesg, journalctl -k, ou /var/log/kern.log
- Counter: Contagem sem log para estatísticas (counter packets 0 bytes 0)
- Rate limiting: limit rate 10/second evita flood de logs
- Log de drops: Essencial para detectar tentativas de ataque
- SIEM: Integração com sistemas de gerenciamento de eventos de segurança
- Análise: Ferramentas como fwlogwatch ou scripts customizados para parse de logs

Práticas de Hardening de Firewalls

- O hardening de firewalls de perímetro envolve configurações que minimizam a superfície de ataque e garantem a integridade das políticas de segurança. A política padrão (default policy) deve ser DROP para todas as chains (input, forward, output), permitindo explicitamente apenas o tráfego necessário. O acesso administrativo deve ser restrito a interfaces específicas (nunca acessível via Internet diretamente) e utilizar autenticação forte (chaves SSH, não senhas). As regras devem ser documentadas extensivamente com comentários. O firewall deve sincronizar seu relógio via NTP para correlação precisa de logs. Revisões periódicas das regras devem remover regras obsoletas e verificar a conformidade com as políticas de segurança atualizadas.

Práticas de Hardening de Firewalls

- Política padrão: DROP all, ALLOW explicitamente o necessário (whitelist)
- Acesso administrativo: Apenas via interface interna ou VPN, nunca via WAN
- Autenticação: Chaves SSH, 2FA, sem login root direto
- Documentação: Comentários em todas as regras explicando propósito
- Sincronização de tempo: NTP essencial para correlação de logs
- Revisões periódicas: Auditoria trimestral de regras e políticas
- Backup: Configurações versionadas e backup antes de alterações

Considerações de Performance

- A performance de firewalls de perímetro é crítica, pois representam um ponto de estrangulamento potencial para todo o tráfego da organização. Firewalls stateful consomem memória RAM proporcional ao número de conexões simultâneas (state table), enquanto firewalls com Deep Packet Inspection (DPI) consomem CPU intensivamente. Para otimização, regras mais frequentemente correspondidas devem ser colocadas no início das chains. O uso de sets (conjuntos) para múltiplos endereços ou portas é mais eficiente que regras individuais. O connection tracking pode ser desabilitado para tráfego específico onde não é necessário (notrack). Em ambientes de alta performance, considere hardware dedicado ou smartNICs com offloading de processamento de pacotes.

Considerações de Performance

- Consumo de RAM: State table cresce com conexões simultâneas (bytes por estado: ~300 bytes)
- Consumo de CPU: DPI e inspeção SSL/TLS são intensivos
- Otimização de regras: Regras comuns primeiro, sets ao invés de múltiplas regras similares
- Notrack: Desabilitar conntrack para tráfego que não precisa (ex: balanceadores internos)
- Offloading: Hardware dedicado para firewalls de alta performance (10Gbps+)
- Benchmarking: Testes de carga regulares para validar capacidade
- Escalabilidade: Clustering de firewalls para redundância e performance

Arquivos da disciplina

- Link para o Google Drive com os arquivos da disciplina:
 - https://drive.google.com/drive/folders/1KuGQgeDWI5RLBZntXmgqlx6NkJnvQx_i?usp=sharing
- Link encurtado:
 - <https://ziply.pk/QbJHLB>